

Logistic Regression

Supervised machine learning algorithm used for classification tasks.

Goal: model the probability of a binary outcome (0 to 1)

Algorithm Flow:

- 1) convert raw text into a numerical representation using BoW or TF-IDF
- 2) linear transformation multiplies each feature by a weight then adds a bias term
- 3) linear combination ($z = wx + b$) is passed through the S-shape sigmoid function
- 4) sigmoid outputs a probability between 0 and 1
- 5) use threshold, often 0.5, for classification

Assumptions:

- Linear Relationship
- No multicollinearity

Loss/Cost Function:

- Log Loss a.k.a. Binary Cross-Entropy Loss

Optimization: Maximum Likelihood Estimation is a broad concept, finding parameters (weights) will make it most likely that I'll see the training data?

Gradient Descent: Batch, Stochastic, or Mini-Batch

Multi-class? Use multinomial logistic regression

When to use:

- linearly separable data
- binary outcomes (yes or no)
- high transparency
- limited compute

Sigmoid Function

Formula:

$$\frac{1}{1 + e^{-z}}$$

SKLEARN IMPLEMENTATION

1) Import Libraries

```
import numpy as np

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.linear_model import SGDClassifier

from sklearn.model_selection import train_test_split
```

2) Split Data

```
notes = [ "patient has chest pain and shortness of breath", "routine checkup no symptoms"]

labels = np.array([1, 0])

notes_train, notes_test, labels_train, labels_test = train_test_split(notes, labels,
test_size=0.3, random_state=29)

vec = TfidfVectorizer()

clf = SGDClassifier(loss="log_loss", random_state=29)
```

3) Train/ Test

```
feats_train = vec.fit_transform(notes_train)

feats_test = vec.transform(notes_test)

clf.fit(feats_train, labels_train)

print(f"err: {1 - clf.score(feats_test, labels_test):.4f}")
```